

AI 底層邏輯結構

X

UIUX 結構系統 - 整合架構

同一個頁面的兩面：人看的介面 與 代理可操作的狀態機

核心命題：在本站，UI 不只是給人看的，它同時是 AI 代理『可讀可寫的狀態機』。一個頁面有兩種觀者--人類（透過 UIUX 結構操作）與光球代理（透過 PageAgent 派發動作）。兩者共用同一份頁面狀態，故必須在同一張藍圖上設計。本文把『AI 底層邏輯結構』與『UIUX 結構系統』各自攤開，再用第 4 章把兩者的每一個接點對起來。

紮根：shared/ 的 orb-*/agent-*/global-agent-*/orb-page-state-graph、client/src 的 design-kit/shells/_shared/PanelState、index.css 設計 tokens、docs/plan/AIDV-118 一致性查核。皆 repo 實檔，非轉述。

1. 整合總覽 - 一張頁面，兩種觀者

把系統想成三條垂直骨架並排，由同一份『頁面狀態 + 頁面註冊表 (APP_PAGE_REGISTRY) 』在底部縫合：

骨架	內容	角色
人類軸 (UIUX)	設計 tokens -> primitives -> design-kit 元件 -> 四殼 -> 頁面；四態/RWD/a11y 紀律	給人操作
代理軸 (AI)	意圖 -> 計畫 -> 安全/成本閘 -> 排程 -> 執行 -> 驗證 -> 自省 -> 感知 -> 重規劃	給代理操作
縫合層 (整合)	APP_PAGE_REGISTRY + orb-page-state-graph + PageAgentContext.dispatch + PageAgentSnapshot	兩軸共用的真相

關鍵洞見：因為兩軸共用同一份頁面狀態，代理的『感知』就是讀 UI 的真實狀態 (PageAgentSnapshot)，代理的『行動』就是派發 UI 動作 (dispatch)。故 UIUX 結構做得好不好，直接決定 AI 能不能可靠地自動操作--這不是兩個專案，是一個。

2. AI 底層邏輯結構（細膩版）

代理軸由內到外四圈：(1)契約層（型別/schema）(2)規劃層（plan/critic）(3)守門層（safety/cost）(4)執行層（schedule/verify/reflect/perceive）。

層圈	模組	內容	目的
契約層	orb-agent-contract / agent-plan-schema / agent-actions	OrbTaskStatus(planned/running/waiting_human/done/failed/cancelled)、16 動作型別 zod、AgentPlan V3、risk low/med/high	一切結構化、可驗證
規劃層	orb-reasoning / orb-plan-critic	planner 抽 intent+限制+引用（思考步驟）-> 產計畫 -> draft->critique->refine 二段精修	執行前把錯攔掉
守門層	agent-plan-safety / orb-cost-governor	風險/能力(hasCapabilityForPage)/已知動作 三檢 + pre-flight 預算闢 + actionRequiresApproval	安全與成本雙紅線
執行層	orb-dag-scheduler / orb-tool-result-verifier / orb-self-reflection / orb-perception-loop / orbTaskOrchestrator	拓撲分批 -> 派發 -> 驗結果 -> 自省寫 orbMemory -> 感知頁面 diff -> 重規劃	做了且確認真的做到

記憶貫穿四圈：Tier A 任務暫存 / Tier B 會話+RAG / Tier C 跨會話 per-spirit 學習。自省的 verdict 寫回 Tier B/C，讓下一輪 planner『讀得到上次教訓』--這是把『學習』接回『規劃』的閉環。

狀態機（orb-task-state-machine）：planned->running->(waiting_human 可暫停等人)->done/failed/cancelled。waiting_human 這個狀態是關鍵--高風險動作或需澄清時，代理會停下來等人，而非硬幹。

3. UI/UX 結構系統（細膩版）

3.1 設計 Token 五層（單一真實來源 = CSS 變數）

色彩真相在 CSS 變數，JS 只存常數/型別。分層：(1)全站 :root (shadcn 語意色換亮色暖光) (2)平台擴充色 (--clay/--gold/--teal/--ok/warn/bad/info) (3)玻璃分層 --surface-1/2/3 (oklch 半透明) (4)暖色陰影 --shadow-healing-sm/md/lg(5)字體 --font-serif (Fraunces/Noto Serif TC) + 圓角 --radius 1rem。主強調 = 黏土/珊瑚橘 #C2613F（也是 --primary）。

Token 範圍化：design-kit 元件用『短別名』（--surface/--text/--muted = 次級文字），與 app 既有 shadcn 同名不同義，故必須包在 .aidv-kit scope 內才正確解析（design-kit.css）--避免污染全站。登入畫面 .login-cosmic carve-out 保留原 cosmic。

3.2 元件階梯：tokens -> primitives -> design-kit -> shells

元件	內容	定位
primitives.tsx	Button/Pill/Tag/Kbd/Eyebrow/Toggle/Meter/Spinner/Card/Input	最小積木，零硬編 hex
states.tsx / PanelState	四態：載入(role=status,aria-busy)/空(role=status)/錯誤(role=alert+重試)/就緒	狀態可視一致
GateCard	確認門卡 + computeGate/countGate（就緒度計算）	成本/風險前的關卡
ShotCard	分鏡卡 + 生成狀態機 frameStyle	影片分鏡的狀態呈現
PromptVault	生成->存庫->重用 閉環（SaveToVault/VaultBrowser）	提示詞資產化
WorkflowBuilder	FlowBar/STEP_LIBRARY/VIDEO_DEFAULT 可設定工作流	把多步流程做成可視積木
orb.tsx	FAB+面板+6 分頁+主動泡泡+人格/心情頭+四態	代理的常駐化身（見第4章）
chrome/cockpit/shellkit	殼層外框/座艙/殼工具	四殼共用骨架

3.3 四殼一脊椎 + 四態鐵律 + RWD/a11y

殼層：ShellFrame 外框 + VideoShell/SocialShell/LearnShell/SettingsShell 四殼，shellRouteTable 統一路由（54 路由保留+轉址+canonical）。脊椎 = Story Spine（左欄 SOX 導航主軸：專案->場景->鏡頭->角色，readiness chip 點了 deep-link 到修復點）。

四態鐵律（AIDV-118 查核）：26 查詢面板 25 有錯誤態出口；首頁 15 framer 動畫 14 守 useReducedMotion；四殼斷點全對齊 --bp 768/1024/1280（與 useMobile 一致）。a11y 基線：全域 :focus-visible、sr-only、四態內建 role/aria-busy/aria-live。（對比度實測/鍵序/真機破版 = Phase 2 人工走查）。

4. 整合接點 - AI 與 UIUX 如何縫在一起（核心）

這是本文重點：代理軸的每個機制，都對應 UIUX 軸的一個具體結構。兩者透過下列接點雙向綁定：

接點	模組 / 元件	作用	具體例子
頁面註冊表	APP_PAGE_REGISTRY	所有頁面/能力的單一登記	代理靠它知道『哪頁能做這動作』
跨頁狀態圖	orb-page-state-graph	由註冊表導出的圖：哪頁處理某 action、到目標頁最短路徑	setModality 只在 /studio、setTab=tts 只在 /pro-studio -> 代理先導航再做
動作派發	PageAgentContext.dispatch	代理把 16 種動作打進真實 React 頁面	fillPrompt->Input、setModel->ModelCard、applyPreset->preset、submit->GateCard
頁面快照	PageAgentSnapshot	代理讀『頁面當前真實狀態』	perception-loop 用前後兩張 diff 判斷『真的變了沒』= UI 即 ground truth
思考可視	orb-reasoning -> 思考步驟面板	planner 的 intent/plan/warnings/citations 攤在 orb 面板	使用者看得到代理怎麼想，不是黑箱
光球心情	OrbMood idle/thinking/working/done	綁代理執行狀態	代理在跑 -> working；做完 -> done；以色彩區分(無臉 calm orb)
四態 ambient	orb states silent/hint/collab/critical	綁代理主張強度	只在關鍵時 critical 介入，平時 silent 不打擾
主動泡泡	orb 主動泡泡(aria-live)	自省/記憶 -> 主動建議	學到的東西(Tier B/C) 變成『下一步去哪』的 CTA
成本閘可視	cost-governor <-> GateCard + 成本階梯儀表	預算閘的決策畫成確認門	高成本 submit 前，使用者看到就緒度+成本再按
脊椎=計畫	Story Spine SOX <-> AgentPlan steps	代理的計畫步驟對映脊椎導航	readiness chip = 該步就緒度；deep-link 到修復點
狀態一致	verifyToolResult / 四態 <-> PanelState	後端 procedure 狀態 <-> 前端四態	載入/空/錯誤/就緒 兩端同義，代理與人看到同一真相

一句話：UIUX 不是『畫面』，是代理的 I/O 介面。把 dispatch (寫) 與 snapshot (讀) 做對，代理才能可靠操作；把四態/readiness/確認門做對，人與代理才看到同一份真相。這就是『整合在一起』的工程意義。

5. 整合架構圖（由上而下，箭頭=資料/控制流）



讀法：人與代理從頂端兩側進入，匯流到『四殼一脊椎』同一套 UI；UI 由 design-kit/primitives/tokens 構成；底部『縫合層』讓代理能讀(snapshot)能寫(dispatch)同一份頁面狀態；再下走到服務與資料層。整條鏈任一環的型別/狀態不一致，AI 自動操作就會破--故型別安全(tRPC/zod)與四態一致是整合的地基。

6. 整合視角：成熟度 · 缺口 · 建議

6.1 已對齊（兩軸縫合良好）

- [+] 動作白名單 + page-state-graph：代理只能做 UI 真有的事，且知道去哪頁做（不會幻想不存在的按鈕）。
- [+] perception-loop 用 PageAgentSnapshot 當 ground truth：UIUX 的真實狀態 = 代理的成功判準，杜絕『以為做到』。
- [+] 四態鐵律 + readiness chip：人與代理共看同一份『載入/空/錯誤/就緒』，狀態語意一致（AIDV-118 查核 25/26）。
- [+] cost-governor <-> GateCard：成本決策被畫成確認門，AI 的預算閘對使用者可見可否決。
- [+] orb 化身：把抽象代理狀態(mood/states/思考步驟/主動泡泡)變成可感知的常駐 UI，黑箱透明化。
- [+] token 範圍化(.aidv-kit) + 旗標化上線：視覺系統可漸進替換、可回滾，不會一次改爆 prod。

6.2 缺口 / 待整合

- [!] orb 視覺實裝多為靜態示範：U-11(AIDV-114) 6 分頁已接真實/前端來源，但語音 Gateway 後端(AIDV-120)未接；部分仍未接 spine。
- [!] Phase 2 人工走查未完：對比度 ≥ 4.5 、焦點環實機、CmdK/updnEnter/Esc 鍵序、真機 RWD 破版 = AIDV-118 待 Bruce 實測。
- [!] page-state-graph 目前最短路徑恆=1 跳：前置條件(需登入/需已上傳資產)尚未建模，複雜流程的代理導航還偏淺。
- [!] 設計 SSOT 與卡描述漂移：AIDV-118 卡早期斷點規格(1181/780...)與實作(768/1024/1280)不符，需回填卡描述。
- [!] 兩軸測試覆蓋不均：代理層(planner/critic/verifier)與四態元件各有測，但『代理真的把某頁操作成功』的端到端整合測較少。
- [!] 新舊光球並存：Bruce 拍板兩種型態同時存在(ENABLE_AIDV_CHROME)，過渡期 UI 一致性需收斂。

6.3 一句話定位

這套系統把『AI 代理』與『UIUX』當成同一個狀態機的讀寫兩面來設計--這在多數產品裡是脫節的（UI 一套、Agent 另接）。本站用 APP_PAGE_REGISTRY/page-state-graph/PageAgent(dispatch+snapshot) 把兩者縫在同一份真相上，並用四態鐵律+確認門+orb 化身讓人與代理共享同一份狀態認知。主要未竟 = 把視覺實裝接滿 spine、補前置條件建模與端到端整合測。